

Abstract

Container terminals must schedule quay cranes jointly with speed-adjustable automated guided vehicles under two conflicting objectives: the makespan of the workload and the energy expended in vehicle travel. No schedule optimises both, so the solution of the problem is a Pareto front of non-dominated compromises from which an operator selects. Methods for computing such fronts are considerably better developed than methods for accounting for them, and the explanatory techniques ordinarily imported from machine learning attribute an outcome to decision variables, whereas the interventions available to an operator are physical.

This thesis attributes the objective values of a Pareto-optimal schedule to the physical activities that produce them. The construction rests on a structural property of the objective: for a fixed schedule the makespan is the longest chain of activities constrained to occur in sequence, so the objective depends upon the activities of that chain alone, and their durations sum to it exactly. The partial derivative of the makespan with respect to any activity duration is consequently zero or one and never an intermediate value, and the resulting attribution balances by construction. Formally the objective is linear over the $(\max, +)$ semiring, which makes the attribution verifiable. The method is a heterogeneous graph attention network, E-HGATv2, which represents a schedule as a typed precedence graph and predicts the duration of each vehicle leg and each crane handling operation; a differentiable max-plus dynamic-programming layer composes those predictions into the makespan, its backward pass routing the subgradient exclusively along the argmax edge of each recurrence, so that differentiating the network returns the critical path itself.

From this decomposition the thesis defines the duration-weighted critical share ρ , the fraction of the makespan spent in vehicle travel, and quantifies the difference between the compromises on a front by the shift in ρ between its two ends. The principal finding is that the binding bottleneck migrates between travel and handling, and that the occurrence of the migration is governed by the fleet-to-crane ratio A/Q . Over 540 replicates spanning six instances, nine fleet sizes and ten seeds, migration is significantly non-zero in 45 of 54 instance–fleet cells under a Holm-corrected Wilcoxon signed-rank test; it attains 0.665 at $A/Q = 2.67$ and vanishes below $A/Q \approx 1$, where both ends of the front are transport-bound. A second campaign of 8,640 replicates reproduces the threshold on three instance families and resolves the attribution at all 657,899 points of the fronts obtained. Where the transfer occurs, cranes bind at the makespan-optimal end and vehicles at the energy-optimal end, which accounts for its absence on the ten published large instances, each of which has $A/Q = 0.50$. The network reproduces the exact attribution to a mean absolute error of 0.026 in the share, with critical-leg agreement of 0.898 on the small instances and 0.924 on the published large ones, and does so at every generation of a search as well as at its termination. That agreement is what extends the method beyond the cases examined here, since the exact recurrence presupposes a schedule that has already been evaluated whereas the network predicts the durations it requires.

Contents

Abstract	1
1 Introduction	3
1.1 Significance	3
1.2 Novelty	4
1.3 Contribution	4
2 Problem Formulation and Background	6
2.1 The problem, its entities and the physics of a leg	6
2.2 Objectives and the completion recurrences	7
2.3 The makespan as a longest path	8
2.4 The multi-population biased random-key genetic algorithm	9
2.5 Selection of the surrogate model class	9
3 Method and Experimental Design	11
3.1 Research question	11
3.2 Graph representation	11
3.3 The E-HGATv2 architecture	12
3.4 The tropical max-plus layer	13
3.5 The duration-weighted critical share	14
3.6 Experimental design	15
3.7 Software architecture and implementation	16
3.8 Reproducibility	17
4 Results	18
4.1 Verification against the source work	18
4.2 A worked example	19
4.3 The degenerate regime	20
4.4 Bottleneck migration across the fleet-to-crane ratio	22
4.5 The three instance families	24
4.6 How the explanation changes during the search	26
4.7 Reproduction by the surrogate	27
4.8 Discussion	28
5 Limitations and Future Work	29
6 Conclusion	30
Bibliography	31

Chapter 1

Introduction

A container terminal mediates between maritime and inland transport, and its throughput is governed by the coordination of its equipment. Quay cranes transfer containers between vessel and quay; automated guided vehicles convey them between quay and storage yard. Scheduling the two resources jointly dominates scheduling them in succession, since an idle crane and an idle vehicle represent respectively unrecovered capital and expended energy. Terminals increasingly deploy vehicles whose travel speed is selectable, which introduces a second objective: a vehicle driven faster completes its leg sooner but consumes disproportionately more energy, power growing super-linearly in speed.

The problem therefore admits no single optimum but a Pareto front: a set of schedules, none of which improves upon another in one objective without conceding the other. Homayouni and Fontes [16] introduce the joint scheduling of quay cranes and speed-adjustable vehicles, and Fontes and Homayouni [11] formulate it as a bi-objective mixed-integer linear program and solve it with a multi-population biased random-key genetic algorithm, reporting fronts for benchmark instances of increasing size. Both contributions concern the efficient computation of the front.

The question that follows has received less attention. An operator is presented with a front and must select from it, and a selection requires knowing what limits each candidate. Two schedules on one front may be limited by different resources: one by vehicles traversing a long yard-to-quay distance, another by a single overloaded crane whose task queue no enlargement of the fleet will shorten. The two cases admit different operational responses, and the objective values alone do not distinguish them. A front records the magnitude of a trade-off without identifying its cause.

1.1 Significance

Whether a front can be computed and whether it can be accounted for are distinct questions, of which the first is settled for this problem and the second is the subject of this thesis. The distinction carries practical content. The interventions available to a terminal operator are physical – enlarging the fleet, rebalancing the task queue of a crane, revising a speed policy – and each addresses a different limiting resource, so a selection made from the objective values alone omits the information the decision requires.

The deficiency is not confined to container terminals. Bengio et al. [4] survey the integration of machine learning with combinatorial optimisation and identify the interpretation of optimiser output as a persistent weakness. The devices customary in multi-objective optimisation are indirect: fronts are visualised, which conveys the shape of a trade-off but not its origin; solutions are clustered, which groups compromises without characterising what separates them; or a preference is elicited and a single solution recommended, which presupposes the understanding it was to supply. Recent work applies post-hoc attribution, principally SHAP [20] and TreeSHAP [21], to surrogate models of the objective. In a scheduling context such attribution lands upon

decision variables, upon positions in a chromosome and discrete speed settings, whereas the operator acts upon *physical quantities*. A chromosome position identifies no vehicle, crane or leg and therefore supports no operational decision, whereas an attribution to a named activity locates both the activity and its cost: in the worked example of Table 4.3, the loaded leg of task 2 contributes 55.556 of the 175.000 seconds of the critical path of L01.

1.2 Novelty

The tractability of the question rests on a structural property of the objective. Rudin [23] argues that where the structure of a model admits an explanation correct by construction, explaining an opaque model after the fact is the inferior alternative; the present problem admits such an explanation. The makespan of a fixed schedule is the greatest total along any chain of activities constrained to occur in order, that is, the length of a longest path through the schedule. Two consequences follow. The objective depends upon the activities of that path alone, so its partial derivative with respect to any activity duration is zero or one; and those durations sum to the makespan exactly. The explanation is therefore read from the structure and balances by construction. Formally the objective is linear over the $(\max, +)$ semiring [2], as developed in Section 2.3.

The two components combined here are individually established. Graph neural networks have been applied to scheduling and routing both as policies [19, 26] and as surrogates, and differentiable optimisation layers embed a combinatorial computation within a network so that gradients traverse it [1, 22], the decision-focused learning literature studying prediction whose loss is defined through a downstream optimiser [9]. Their combination for this problem class is, to the author’s knowledge, not established. The novelty lies in the quantity the network is required to predict. It does not predict the makespan. It predicts the local physical quantities of a schedule, the duration of each vehicle trip and each crane lift, which a max-plus layer composes into the makespan by the same recurrence the exact evaluator employs. Differentiating the network with respect to its own predictions therefore recovers the critical path itself, and the result is expressed in seconds attributable to named activities rather than in units of an internal representation.

1.3 Contribution

The thesis advances a single proposition: *that a Pareto-optimal schedule admits an exact explanation in seconds attributable to named physical activities, and that a learned model can produce the same explanation*. Every result reported below is subordinate to that proposition.

The explanation is exact rather than estimated. For a given schedule the method identifies which vehicle trips and crane lifts determine the makespan, and how many seconds each contributes. Because those durations sum to the makespan exactly, the explanation admits verification; it is verified on every schedule reported, so that an inconsistency would surface as a failed test. The property holds because the makespan of a fixed schedule is the longest chain of activities constrained to run in sequence, which restricts the sensitivity to any one activity to zero or one. The learned model, E-HGATv2, is not asked to predict the makespan. It predicts the duration of each trip and each lift, and a differentiable max-plus layer composes those into the makespan

by the recurrence the exact evaluator uses. Differentiating the model with respect to its own predictions therefore returns the critical path itself, expressed in the same physical units. Its agreement with the exact result is measured throughout and reported in Section 4.7. That agreement is what carries the method beyond the instances examined here, since the exact recurrence presupposes a schedule that has already been evaluated whereas the model predicts the durations it requires.

That the explanation is worth obtaining is established by one empirical fact: the answer is not uniform along a front. Given a sufficient fleet, the makespan-optimal end of a front is limited by the cranes and the energy-optimal end by the vehicles, so two schedules between which an operator is choosing are limited for different reasons and admit different interventions. Whether this occurs is governed by the number of vehicles per crane, which is established by a statistically tested sweep (Section 4.4) and reproduced on three instance families. It is offered as evidence for the proposition: without it, explaining a schedule would demonstrate the method on one solution while establishing nothing about the front.

Chapter 2 states the problem, its formulation, the solution method and the relevant background. Chapter 3 describes the surrogate architecture, the max-plus layer, the derived measure and the experimental design. Chapter 4 evaluates the results, beginning with verification. Chapter 5 states the limitations. Chapter 6 concludes.

Problem Formulation and Background

2.1 The problem, its entities and the physics of a leg

An instance is a set of container moves (*tasks*) $\mathcal{J} = \{0, \dots, N-1\}$, each bound to one quay crane $qc(j)$ with handling time τ_j and one yard station $lu(j)$, served by a fleet of $|\mathcal{A}|$ speed-adjustable vehicles. A task is of one of two kinds. A *loading* task ($j \in \mathcal{L}$, a LOAD) moves a container from yard to vessel: a vehicle travels empty to the yard station, collects the container, carries it to the crane, and the crane lifts it aboard, so its loaded leg runs $LU \rightarrow QC$. An *unloading* task ($j \in \mathcal{U}$, an UNLOAD) reverses this, the crane lifting a container from the vessel onto a waiting vehicle which transports it to the yard. Each crane processes its tasks one at a time.

A decision specifies, for every task, the assigned vehicle, its position in that vehicle's sequence and in its crane's sequence, and a discrete *speed level* for the empty leg and for the loaded leg, one of a small fixed set of velocities at which a vehicle may be driven; a higher level shortens the leg and raises its power draw. The instances follow the three-level configuration of Fontes and Homayouni [11]. Two objectives are minimised: the makespan $C_{\max}$, the completion time of all work, and the energy E consumed by vehicle travel. Crane handling energy is excluded from E , which is material to the interpretation of both objectives, since the crane contributes to the makespan alone.

Table 2.1 collects the notation. Sequencing is represented by the order of tasks within a resource's chain, and the formulation below states the problem as the timing recurrences a resolved schedule satisfies.

The recurrences of Section 2.2 are the form in which the implementation computes the objective, and the form in which the max-plus structure of Section 2.3 is manifest. The mixed-integer programme of Fontes and Homayouni [11] formulates the same problem and supplies the published optima used for verification in Chapter 4.

Physics of a leg. Three speed levels, with multipliers $\alpha_\ell \in \{0.8, 1.0, 1.2\}$, scale a nominal speed; empty and loaded legs have distinct nominal speeds and power draws (Table 2.2). For a leg of length d travelled at level ℓ ,

$$t_{\text{leg}}(d, \ell) = \frac{d}{v_\ell}, \quad e_{\text{leg}}(d, \ell) = P_\ell \cdot t_{\text{leg}}(d, \ell), \quad (2.1)$$

so energy is power times time. A faster level shortens t but raises P , and this is the sole source of the time–energy trade-off.

Table 2.1: Notation. The upper block is problem data, the middle block schedule variables, and the lower block quantities introduced by this thesis. The three speed levels and their constants are given; which level each leg runs at is part of the decision, as Section 2.1 states.

Symbol	Meaning
$\mathcal{J}, N$	set of container moves (tasks), $N = \mathcal{J} $
$\text{qc}(j), \text{lu}(j)$	the crane and yard station to which task j is bound
$\text{qcprev}(j)$	the predecessor of j in its crane's sequence
$Q, \mathcal{A} $	number of quay cranes and of vehicles
τ_j	quay-crane handling time of task j
ℓ	one of three settings at which a leg may be driven; selected per leg
$\alpha_\ell, v_\ell, P_\ell$	speed multiplier, speed and power draw at level ℓ (Table 2.2)
d_k	length of leg k ; k indexes legs, never speed levels
$t_j^{\text{empty}}, t_j^{\text{loaded}}$	empty and loaded leg durations of task j
$e_j^{\text{empty}}, e_j^{\text{loaded}}$	empty and loaded leg energies of task j
a_j^{agv}, c_j	vehicle-ready time and completion of task j
$C_{\max}, E$	makespan and total vehicle energy
$\chi \in [0, 1]^{4N}$	random-key chromosome
G_s	heterogeneous schedule graph, arc types $E^{\text{agv}}, E^{\text{qc}}$
h_j	node embedding produced by the E-HGATv2 core
$w[v], w[u, v]$	node and edge weights of the max-plus dynamic programme
$g_a = \partial C_{\max} / \partial w_a$	attribution of activity a ; exactly 0 or 1
π^*	the critical (longest) path
$\rho_{\text{transport}}$	duration-weighted critical share, Eq. (3.2)
A/Q	fleet-to-crane ratio, the sweep parameter

Table 2.2: Kinematic and power constants at each speed level ℓ , after Fontes and Homayouni [10]. These are the authors' own three-velocity scenario and are used unchanged. Each row fixes ℓ , so the subscript is supplied by the row; the symbols are those of Table 2.1.

Level ℓ	α_ℓ	empty v_ℓ (m/s)	empty P_ℓ (kW)	loaded v_ℓ (m/s)	loaded P_ℓ (kW)
Lower	0.8	4.8	7.8	2.4	11.7
Nominal	1.0	6.0	10.0	3.0	15.0
Higher	1.2	7.2	13.2	3.6	19.8

2.2 Objectives and the completion recurrences

Total energy is additive over the chosen legs,

$$E = \sum_{j \in \mathcal{J}} (e_j^{\text{empty}} + e_j^{\text{loaded}}). \quad (2.2)$$

The makespan requires the precedence structure. Fixing the assignment and the sequencing turns the schedule into a precedence graph: task j cannot proceed until both its predecessor on the same vehicle and its predecessor on the same crane have finished. Writing a_j^{agv} for the time at which the vehicle assigned to j becomes free for it, and $\text{qcprev}(j)$ for j 's predecessor on its crane, the per-task completion obeys, for a LOAD,

$$c_j = \max(a_j^{\text{agv}} + t_j^{\text{empty}} + t_j^{\text{loaded}}, c_{\text{qcprev}(j)}) + \tau_j, \quad (2.3)$$

and for an UNLOAD,

$$c_j = \max(a_j^{\text{agv}} + t_j^{\text{empty}}, c_{\text{qcprev}(j)} + \tau_j) + t_j^{\text{loaded}}. \quad (2.4)$$

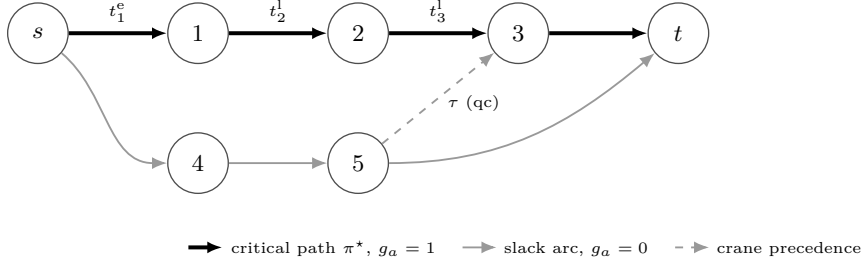


Figure 2.1: The event DAG of a small schedule. Solid arcs are vehicle legs and the dashed arc a quay-crane serialisation constraint. The bold path $s \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow t$ is the critical path π^* ; by Section 2.3 only its arcs carry a non-zero attribution, and their durations sum to $C_{\max}$ by Eq. (2.6).

The makespan is then

$$C_{\max} = \max_{j \in \mathcal{J}} c_j. \quad (2.5)$$

The asymmetry between (2.3) and (2.4) is an essential feature. For a LOAD the handling τ_j is added after the maximum, because the crane must lift the container once the vehicle has delivered it; for an UNLOAD the handling appears within the maximum on the crane branch only, because the crane’s lift overlaps the vehicle’s travel to the quay, and the loaded leg is then added afterwards. A LOAD therefore completes when its crane finishes, whereas an UNLOAD completes when its vehicle reaches the yard. Chapter 4 shows that this asymmetry is precisely what reconciles the single-maximum objective with the two-array formulation used in the reference implementation.

2.3 The makespan as a longest path

Each constraint above asserts that a completion time is at least the sum of a predecessor and a duration. At optimality every such variable equals the greatest of its lower bounds. Writing $\oplus$ for maximum and $\otimes$ for addition, the schedule is a linear system over the $(\max, +)$ semiring [2, 14], and $C_{\max}$ is the length of a longest path in the directed acyclic graph whose vertices are activities (empty legs, loaded legs and crane handlings) and whose arcs are the precedences above. The construction is the disjunctive graph familiar from job-shop scheduling [6], specialised to two resource types.

Two consequences follow, upon both of which the thesis depends.

First, the objective depends upon the activities of one path alone. Perturbing the duration of an on-path activity alters $C_{\max}$ by the same amount, whereas perturbing an off-path activity leaves $C_{\max}$ unchanged until the perturbation becomes large enough to displace the longest path. The sensitivity is therefore piecewise constant, equal to one on the critical path and zero elsewhere. Second, as an immediate corollary, the durations of the on-path activities sum to the makespan:

$$C_{\max} = \sum_{\text{on-path legs } k} d_k + \sum_{\text{on-path tasks } j} \tau_j. \quad (2.6)$$

This is an identity, not an approximation, and it is the property that the architecture of Chapter 3 is designed to preserve. Figure 2.1 illustrates the construction on a small schedule.

2.4 The multi-population biased random-key genetic algorithm

Fronts are produced by the mp-BRKGA of Fontes and Homayouni [11]. Following the biased random-key paradigm [13], a solution is a vector of $4N$ keys comprising one block of task priorities, two of empty- and loaded-leg speed levels and one of vehicle assignments, which a decoder maps to a schedule that is then evaluated exactly. The method evolves $\Omega + \Pi$ populations concurrently: each single-objective population ranks by one objective and so drives one extreme of the front, while each multi-objective population ranks by non-dominated sorting with crowding distance [8]. Each generation replaces a population by its elite fraction, offspring obtained from biased parameterised uniform crossover inheriting from the elite parent with probability $p_e = 0.7$, and fresh mutants; elites migrate between populations so that boundary specialists propagate inward.

The published configuration is $P = 20N$ per population, elite fraction 0.2, mutant fraction 0.1, $p_e = 0.7$, $\Omega = \Pi = 2$ and $n_{ex} = 30$, adopted here with the reduced generation budget of Section 3.6. The search is not a contribution of this work; it is the instrument supplying Pareto-optimal schedules to be explained, and no learned model participates in producing any front reported here.

2.5 Selection of the surrogate model class

Exact evaluation costs well under a millisecond at these sizes, so the model is not introduced in order to render the search tractable. Its purpose is that a model predicting the per-activity structure of a schedule may be differentiated with respect to its own predictions, which yields an explanation derived from a learned representation rather than from the exact recurrence and permits the two to be compared. The exact recurrence presupposes a schedule whose durations are already resolved; the model predicts them, and therefore applies wherever that presupposition fails.

Three properties of the input determine the model class. First, a resolved schedule is a typed precedence graph, not a vector. Any model consuming a fixed-length feature vector must serialise it as the $4N$ chromosome of Section 2.4, as a tree ensemble such as XGBoost [7] would; that representation has a length depending on N , so it does not transfer across instance sizes, and its coordinates are decision-variable indices, so any attribution over it, TreeSHAP [21] included, attributes to chromosome positions rather than to physical activities. Second, the recurrences (2.3)–(2.4) relate a task to its immediate predecessor on a resource, which is message passing [12]: a graph network is constrained to the locality a multilayer perceptron would have to learn. Third, the two relations enter (2.3) and (2.4) differently and therefore cannot share parameters, and the quantities that matter, travel time and leg energy, are properties of arcs, which forces edge conditioning.

These jointly specify a heterogeneous, edge-conditioned message-passing network. The attentional variant is adopted in the corrected GATv2 form of Brody et al. [5] in preference to the original GAT [24], whose attention is monotonic in the key and therefore cannot rank neighbours conditioned on the query. Cross-relation fusion follows the semantic attention of Wang et al. [25]; Section 3.3 explains why that cross-relation attention, and not the within-relation

attention, carries the informative signal here.

Method and Experimental Design

3.1 Research question

The investigation asks whether, for a schedule on the Pareto front of the problem of Chapter 2, one can state exactly why it is efficient, and whether that explanation varies systematically along the front. The first part is addressed constructively by the architecture below; the second is empirical and requires a scalar summary of an explanation.

3.2 Graph representation

Figure 3.1 gives the pipeline as a whole; this subsection and the two that follow describe its three stages.

A resolved schedule is encoded as a heterogeneous graph. Nodes are tasks, carrying four features: the handling time τ_j , two indicators distinguishing loading from unloading, and the identifier of the servicing crane. Arcs are of two types. An *AGV arc* connects consecutive tasks on the same vehicle; a *QC arc* connects consecutive tasks on the same crane. Each arc carries three features: the total travel time of the leg it represents and the empty and loaded energies of that leg.

The graph is therefore precisely the precedence DAG of Figure 2.1, so the representation is isomorphic to the combinatorial structure of the objective. Two aspects are consequential: each task has at most one incoming arc of each type, with the architectural implication of Section 3.3, and travel time and energy reside on arcs, so the model must condition its messages on edge attributes as well as on node states.

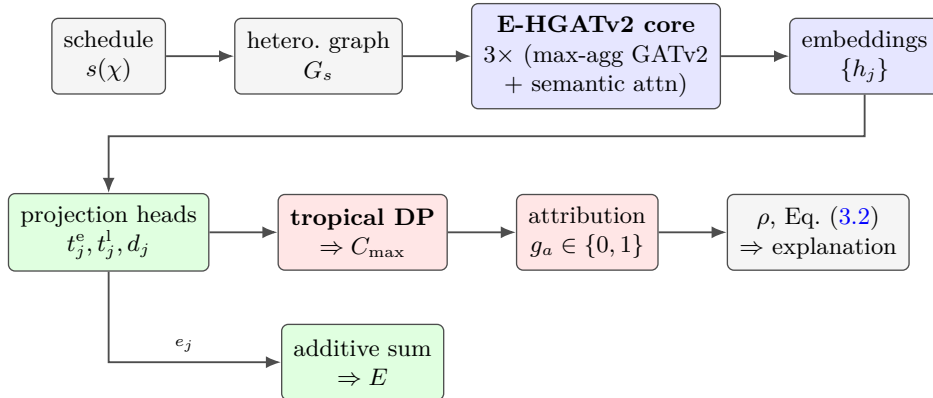


Figure 3.1: The pipeline. The core embeds a decoded schedule; projection heads predict local physical attributes, which an exact max-plus dynamic programme composes into $C_{\max}$, energy being an exact additive sum. Differentiating the dynamic programme yields the binary attributions $g_a = \partial C_{\max} / \partial w_a$, from which ρ is computed. The network never regresses an objective directly.

3.3 The E-HGATv2 architecture

Each layer updates the fixed-length description held at every task by combining it with those held at its immediate predecessors on its vehicle and on its crane, so stacking L layers lets information reach a task from predecessors up to L positions away in either chain. The surrogate has three such layers, a hidden dimension of 64 and four attention heads per layer, and each layer performs two operations.

Attentional aggregation. A layer must combine several incoming messages into one. The simplest such combination weights them uniformly. *Attention* instead learns the weights [3]: for each candidate it scores the pair of descriptions at the two ends of the connection, normalises those scores across the candidates being combined so that they sum to one, and returns the weighted average; a *head* is one such weighting, and computing four in parallel and concatenating them lets a layer attend on four different grounds at once. The weights are thus a learned answer to which of the available candidates matters for this task, recorded in a form that can afterwards be inspected. Two properties of the normalisation bear on the argument that follows. It is taken over a particular set of candidates, so what an attention is informative about is fixed by what that set contains; and when the set holds a single candidate its weight is forced to one, carrying no information whatever. This network attends twice per layer, over two different sets, and only one of the two escapes that degeneracy.

Within-relation propagation under a max-plus reduction. For each relation $R \in \{\text{AGV}, \text{QC}\}$ separately, a GATv2 convolution [5] conditioned on the arc attributes computes a message for every node from its predecessors under R . The aggregation is **max** rather than the customary attention-weighted sum. This choice mirrors the semiring: the recurrences of Section 2.3 reduce a node’s same-resource predecessors under a maximum, and the aggregator is therefore selected to match the algebra of the problem.

Cross-relation fusion by semantic attention. The two relation messages are combined by a learned attention over relations, in the manner of Wang et al. [25]. This is where the informative signal resides, and the reason is a direct consequence of the encoding of Section 3.2. Since a resolved schedule gives each task at most one predecessor per resource, the within-relation attention softmax is computed over a single incoming arc and is therefore degenerate, equal to one identically. It contributes no discriminative information. The non-degenerate quantity is which of the two resources gates the task, exactly the $\max(\text{agv_ready}, \text{qc_ready})$ argmax of the recurrence, and this is what the cross-relation attention represents. Its per-node weight on a relation is interpretable as the criticality of the corresponding incoming arc, and it is recorded per arc for that purpose.

This asymmetry inverts the conventional reading of an attentional graph network, in which the within-neighbourhood attention constitutes the object of interest. In the present setting that component is structurally vacuous, and the cross-relation component carries the resource-contention signal. Figure 3.2 shows one layer.

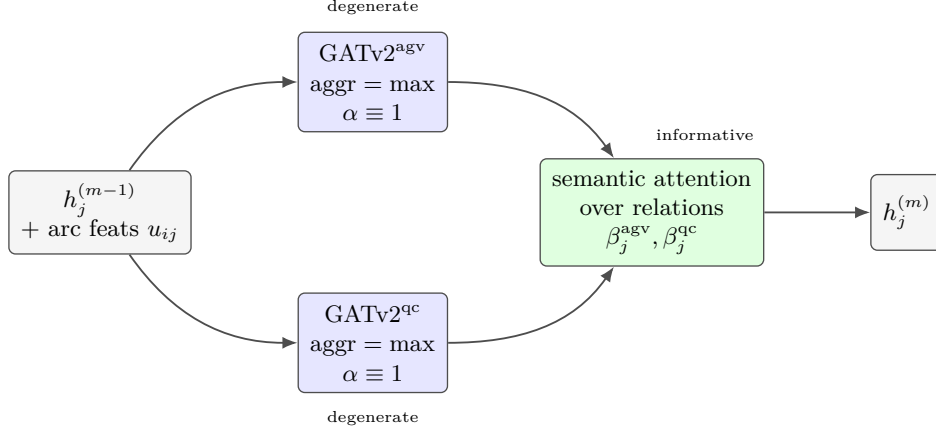


Figure 3.2: One heterogeneous layer. Message passing runs independently per relation under a maximum aggregation, and the two relation messages are then fused by a learned attention across relations. Within a relation a resolved schedule gives each task at most one incoming arc, so the GATv2 softmax is over a single arc and is degenerate ($\alpha \equiv 1$). Across relations β_j is non-degenerate and encodes which resource gates task j , the $\max(\text{agv_ready}, \text{qc_ready})$ argmax of the recurrence; it is recorded per arc as that arc’s criticality. The informative attention is therefore the cross-relation one.

3.4 The tropical max-plus layer

The architecture described so far could be completed in the conventional manner, by pooling node embeddings and regressing $(C_{\max}, E)$ with a multilayer perceptron. That choice would forfeit the property the method depends on. A pooled regression head mixes every node embedding into every output, so the Jacobian of the predicted makespan with respect to the input features is dense. An attribution read from it distributes credit across activities that are provably irrelevant, because Section 2.3 establishes that only the activities of one path can affect $C_{\max}$ at all. The head, not the encoder, is what destroys faithfulness.

The makespan head is therefore replaced by a differentiable max-plus dynamic programme. The layer computes, for a directed acyclic graph of activity events,

$$x[v] = w[v] + \max_{u \rightarrow v} (x[u] + w[u, v]), \quad (3.1)$$

in a deterministic topological order obtained by Kahn’s algorithm, with node weights $w[v]$ and edge weights $w[u, v]$ supplied by the network. The makespan is the maximum over sinks. The layer is implemented as a custom autograd operation: the forward pass records the maximising predecessor of each node, and the backward pass routes the incoming subgradient exclusively through that recorded argmax edge.

The consequence is the property sought. Because the backward pass follows a single path, $\partial C_{\max} / \partial(\text{activity duration})$ is exactly one for activities on the critical path and exactly zero otherwise. Attribution is not estimated; it is read off a selection that the forward pass already made. Figure 3.3 contrasts this with the pooled head it replaces.

Equation (2.6) then holds for the model’s own predicted durations exactly as it holds for the exact ones, so that the model’s explanation and the exact explanation may be compared on identical terms. The layer belongs to the family of differentiable optimisation layers [1, 22], and is distinguished by the fact that the embedded computation is a dynamic programme whose subgradient is available in closed form, where such layers ordinarily embed an optimisation

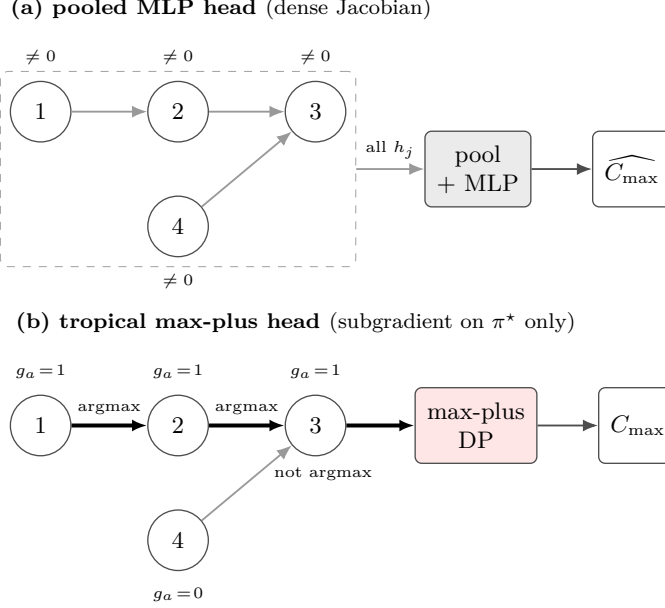


Figure 3.3: The output head, not the encoder, determines whether the attribution is faithful. Both panels encode the same four activities, three on the longest path while activity 4 loses the maximum at node 3. (a) A pooled regression head mixes every node embedding into the output, so every activity receives a non-zero derivative, including activity 4, which provably cannot affect $C_{\max}$. (b) The max-plus layer records the maximising predecessor during the forward pass and routes the subgradient through it alone, so the attribution is the indicator of the critical path.

requiring implicit differentiation.

Ties in (3.1) are resolved by the first maximum in edge order, following the deterministic **argmax** convention. This is a valid subgradient selection, since any single maximiser of a maximum is a subgradient. Chapter 5 states the consequence for schedules on which the maximum is attained more than once.

Prediction targets, anchoring and the energy head. The network does not predict the objectives. An arc-level head maps the concatenated endpoint embeddings of a task’s incoming AGV arc to its empty and loaded travel times and leg energies, and a node-level head maps a task embedding to its crane handling delay; the makespan is composed from these by (3.1) and the energy by exact summation, a linear head for which $\partial E / \partial e_k = 1$ identically. Because a max-plus output constrains only the attributes on the currently active path, training on the makespan alone would leave off-path attributes unidentified, so the local attributes are supervised directly against their exact physical values. Training is therefore two-stage: the core is trained first, then frozen with cached embeddings while the heads are fitted. One consequence should be read with care. The arc features the encoder aggregates include the per-leg energies themselves, so predicting total energy is close to reading a sum of inputs, and the near-unit coefficient of determination in Section 4.7 is a property of the architecture and should not be read as evidence of learning.

3.5 The duration-weighted critical share

Comparing explanations across schedules requires a scalar. The natural candidate is the proportion of the critical path attributable to vehicle travel.

Counting on-path activities of each kind is inadequate. Legs and handlings are not commensurable: a task may contribute a critical handling while its vehicle chain also binds, so the two counts neither partition the path nor sum to a meaningful total. An earlier formulation of this work used counting and produced a measure that failed to reconcile with the makespan.

The measure adopted here weights by duration. Define

$$\rho_{\text{transport}} = \frac{\sum_{\text{on-path legs}} d_k}{\sum_{\text{on-path legs}} d_k + \sum_{\text{on-path tasks}} \tau_j}, \quad \rho_{\text{handling}} = 1 - \rho_{\text{transport}}. \quad (3.2)$$

By (2.6) the denominator equals $C_{\max}$ exactly, so the two shares partition the makespan and sum to unity by construction. Neither term counts objects, which removes the commensurability problem entirely.

The quantity of interest is the shift in $\rho_{\text{transport}}$ between the two ends of the front,

$$\text{migration} = \rho_{\text{transport}}(\text{makespan-optimal}) - \rho_{\text{transport}}(\text{energy-optimal}). \quad (3.3)$$

The sign is retained rather than taken in absolute value, because it identifies which end is travel-heavy. A negative value indicates that the energy-optimal end leans more on travel than the makespan-optimal end does, so that prioritising energy moves the bottleneck towards vehicle travel; a positive value indicates the converse; and a value near zero indicates that the character of the bottleneck is invariant along the front even though the objective values are not.

3.6 Experimental design

Instances. Two published sets are used directly [10, 11], and a further two families are constructed from the first as Section 4.5 describes. The small set comprises 35 loading instances of 4 to 16 tasks on a six-crane, six-station layout. The large dual-cycling set comprises 10 instances of 40 to 160 tasks on a twenty-crane, twenty-station layout; it is referred to throughout as the large dual-cycling set, to distinguish it from the 26 small mixed instances of Section 4.5, which are dual-cycling in the same operational sense but are a different set on a different layout. Both were transcribed programmatically from the published tables and the authors’ solver sources; no value was retyped. The transcription was cross-validated: the distance matrix and task lists were parsed independently twice and agreed in every cell, and four published optima and five mixed-instance pairings matched exactly.

Fleet sweep. The parameter of interest is the fleet-to-crane ratio A/Q . Because every published large instance has $A/Q = 0.50$, the published sets cannot by themselves determine whether migration depends on this ratio. Six small instances (L01, L07, L15, L21, L28, L35) were therefore each run at nine fleet sizes, $A \in \{1, \dots, 9\}$, with ten seeds: 540 replicates, all other parameters fixed.

Front campaign. The sweep above varies the ratio on six loading instances and records the transport share at the two extremes of each front. A second campaign widens both axes. It covers all 96 instances of the three families of Section 4.5 at the same nine fleet sizes and ten

seeds, giving 8,640 replicates, and computes the share at every point of every front rather than at its extremes, giving 657,899 explained schedules. No surrogate is fitted in this campaign: every quantity is produced by the exact evaluator and the exact attribution, so a replicate costs a few seconds and the whole campaign runs in about ten minutes on 200 cores.

Budget and statistics. Each replicate runs mp-BRKGA at $P = 20N$ per population with $\Omega = \Pi = 2$, $n_{ex} = 30$ and 100 generations, below the published 300; since no comparison is against a published front and every arm shares the budget, the reduction affects absolute front quality but not the migration measurement. Migration is tested against zero by a one-sample Wilcoxon signed-rank test across the ten seeds of each instance–fleet cell, with p -values Holm-corrected [15] within each instance’s nine fleet sizes so that nine simultaneous tests cannot produce a spurious rejection. Effect size is the matched-pairs rank-biserial correlation [18]. A non-parametric procedure is used because ten observations per cell do not support a distributional assumption and several cells are degenerate with near-zero variance. Front quality, where reported, uses hypervolume [27] and IGD^+ [17].

3.7 Software architecture and implementation

The implementation is a Python package of 50 modules and approximately 10,200 source lines, organised so that the exact physics, the learned model and the search are separable and independently testable. The `environment` package is the exact physics and carries no learned component; `surrogate` holds the graph construction and the encoder; `explain` holds the tropical layer, the fused head and both explainers; and `baselines`, `metrics` and `benchmark` form the evaluation harness.

The exact evaluator as label source. There is no external dataset. Every training target is produced by the exact evaluator, which runs (2.3)–(2.5) in topological order in $O(N)$ and returns both the objective pair and the per-task quantities the projection heads are supervised on. Random-key vectors are sampled uniformly, decoded and evaluated, so the data is generated self-supervised. The physics anchoring of Section 3.4 depends on this, and it is why the evaluator’s agreement with the source work is a precondition for every other result.

The autograd function. The max-plus layer is a `torch.autograd.Function` rather than a composition of differentiable primitives, because the required backward behaviour is not the one automatic differentiation would supply for a chain of `torch.maximum` calls. The forward pass establishes a deterministic topological order by Kahn’s algorithm, rejecting any input that is not acyclic, since the precedence graph of a feasible schedule must be a DAG. It then relaxes (3.1) in that order, and stores the maximising predecessor of every node. The backward pass then walks the stored selections in reverse, accumulating the incoming gradient into exactly one predecessor per node. The `argmax` used in the forward pass is what implements the tie convention: it returns the first maximal index, so a tie yields one valid subgradient, and never a blend of several. The same convention is applied at the terminal reduction over sinks, for the reason Chapter 5 gives.

Assertions, testing and execution. Node and arc feature tensors are validated at every entry point against declared semantic signatures, and the decomposition identity (2.6) is asserted on every traversal at a relative tolerance of 10^{-6} , so an inconsistency raises instead of being averaged into a table. The suite comprises 29 unit-test modules and 239 tests, all passing, including the equivalence of the max-plus evaluator with an exhaustive enumeration of all 1248 schedule structures on a five-task instance and the mp-BRKGA operator semantics against the authors’ source. Campaigns ran on a rented cloud instance of 384 vCPU and 125 GB RAM under Python 3.12 and PyTorch 2.11, distributed over a process pool; the 540-replicate sweep took 47 minutes and the 350-replicate small-set campaign 10 minutes, neither with a single failure.

3.8 Reproducibility

Every table in Chapter 4 is generated programmatically from the stored experiment output, so no figure in this document is transcribed by hand. Replicates are seeded and isolated: a failure in one cannot affect another, and failures are counted in the output. Every replicate of every campaign ran to completion: no run crashed, timed out or was lost, over the 940 replicates of the sweeps and the 8,640 of the front campaign alike. The decomposition check of Chapter 5 is satisfied as widely: the recovered critical path reconciles with the makespan on all 940 replicates and at all 657,899 front points, so no quantity reported in this document is computed on a filtered subset.

Results

4.1 Verification against the source work

Before any explanation may be trusted the evaluator must agree with the source work. Fontes and Homayouni [11] compute the objective as $\max(\max_j c_j, \max_a A_a)$, in which A_a tracks when vehicle a next becomes available, whereas the implementation here takes a single maximum over per-task completions. The expressions are identical, and the reason is the asymmetry of (2.3) and (2.4): for a LOAD the availability equals $c_j - \tau_j$, the vehicle being released as soon as the crane assumes the container, and is dominated by $\max_j c_j$; for an UNLOAD it equals the completion of the loaded leg, which by (2.4) is exactly c_j . The reference implementation distributes across two arrays what (2.5) folds into one. On 2400 random schedules across eight instances of both sets the two expressions agreed to exactly zero difference in every case.

A second, independent check compares the makespan-optimal end of the computed fronts against the mixed-integer optima published for all 35 small instances [10]. This is not a test of exact agreement: the published optima are computed for fixed-speed scenarios, whereas the present runs select among three speeds per leg and therefore solve a less constrained problem whose makespan should lie at or below any fixed-speed optimum. A value above one would indicate an evaluator error or a search shortfall; a value below is the expected consequence of the enlarged decision space.

Table 4.1 reports the comparison. Across all four published makespan columns the best computed makespan lies at or below the published value in effectively every comparable cell, the two exceptions being L03 and L25, which exceed the nominal-higher column by 0.56 and 1.56 seconds respectively.

The loading family is Table 5 as printed, but the unloading and mixed families are reconstructions, so each is checked against the optima published for it. Table 4.2 reports both.

The closest correspondence is with the normal-higher scenario, at a mean gap of -16.16 seconds, and on L01 the agreement is exact: the makespan-optimal end is 175.00 against a published 175. That this is the nearest column is the expected outcome, since that scenario permits the

Table 4.1: The makespan-optimal end of the computed fronts against the published fixed-speed optima, over the 35 small instances. The present runs select among three speeds per leg and therefore solve a less constrained problem, so the computed value should lie at or below each published optimum; a value above one would indicate an evaluator error or a search shortfall. The nominal-higher scenario is the nearest comparison because it permits the highest fixed speed. 14 published cells are blank, the authors’ solver having failed to close them, and are skipped rather than imputed.

Published column	comparable	at or below	exact	mean gap (s)
nominal, $C_{\max}$	35	35	0	-365.50
nominal, $C_{\max}^*$	35	35	0	-98.45
lower-nominal, $C_{\max}$	33	33	0	-529.76
nominal-higher, $C_{\max}^*$	23	21	1	-16.16

Table 4.2: The two reconstructed instance families against the mixed-integer optima published for them, at the published fleet size of two vehicles. The unloading family reverses the origins and destinations of Table 5; the mixed family combines the pairs named in the “comb. of” column of Table 3, and is the 26-instance small set, not the ten large dual-cycling instances of Table 4.5. Ratios below one are expected, since the runs here select among three speeds per leg where the published optima fix one, so they solve a less constrained problem. The final column counts instances whose energy optimum is reproduced exactly, which the speed selection cannot improve upon and which therefore tests the reconstruction directly.

Family	instances	$C_{\max}$ ratio		E ratio	E exact
		mean	range	mean	
unloading	35	0.915	0.740–0.975	0.996	18/33
mixed	26	0.935	0.793–1.030	0.960	11/25

Table 4.3: Worked example: the exact makespan critical path of the makespan-optimal Pareto schedule of L01 ($N = 4$, $A = 2$, $Q = 2$). The on-path durations sum to $C_{\max} = 175.000$ exactly, because every on-path activity carries a unit subgradient. No surrogate is involved in producing this table.

Order	Task	Activity	Duration (s)	$\partial C_{\max}$	Resource
0	t_2	empty_leg	0.000	1	AGV 1, QC2
1	t_2	loaded_leg	55.556	1	AGV 1, QC2
2	t_1	empty_leg	27.778	1	AGV 1, QC1
3	t_1	loaded_leg	41.667	1	AGV 1, QC1
4	t_1	qc_handling	50.000	1	AGV 1, QC1
Sum of on-path durations			175.000		$= C_{\max}$
of which travel / handling			125.000 / 50.000		$\rho = 0.714$

highest fixed speed and is therefore the fixed-speed problem closest to the present one. The check establishes that the evaluator operates in the correct regime and never produces an implausibly low makespan against an independently computed bound; it does not, and cannot, reproduce the published values.

4.2 A worked example

Table 4.3 presents the complete exact explanation of a single schedule: the makespan-optimal Pareto point of L01, the smallest instance and therefore verifiable by hand. The critical path comprises five activities whose durations sum to the makespan exactly, as (2.6) requires.

The traversal exhibits two properties. Every on-path activity belongs to the same vehicle, which establishes that the schedule is vehicle-limited and that the pertinent intervention is fleet capacity. The division between travel and handling is quantified rather than asserted: $\rho = 0.714$ indicates that slightly over seventy per cent of the makespan consists of vehicle movement and the remainder of crane handling. This constitutes the form of statement this thesis aims to produce.

That table explains one schedule. The claim the rest of this chapter makes is about a pair of them, so Table 4.4 gives both ends of one front at the same granularity. The two critical paths of L21 differ in kind, not in length: four of the ten activities binding the makespan-optimal schedule are crane handlings, against one of fifteen at the energy-optimal end, where the path is almost entirely vehicle travel. The aggregate measure summarises the difference between the two blocks. Prioritising energy slows every leg, which lengthens travel until it dominates a path that the cranes had previously gated, and the share moves from 0.464 to 0.901 in consequence. Section 4.4 establishes that this happens systematically; the table shows what is being counted.

Table 4.4: The two ends of one front, activity by activity, for L21 ($N = 12$, $A = 2$, $Q = 3$, $A/Q = 0.67$, seed 0). Each block is a complete critical path: every listed activity carries a unit subgradient, and the durations sum to that end’s makespan exactly. The two differ in kind, not degree. 4 of the 10 activities binding the makespan-optimal schedule are crane handlings, against 1 of 15 at the energy-optimal end, so the share moves from 0.464 to 0.901. That shift is the migration of Table 4.6, here read off named activities rather than averaged over seeds. No surrogate is involved.

Order	Task	Activity	Duration (s)	$\partial C_{\max}$	Resource
<i>makespan-optimal end: $C_{\max} = 434.39$ s, $E = 13,558$ kJ</i>					
0	t_5	empty_leg	0.0	1	AGV 1, QC2
1	t_5	loaded_leg	55.6	1	AGV 1, QC2
2	t_{10}	empty_leg	20.8	1	AGV 1, QC3
3	t_{10}	loaded_leg	55.6	1	AGV 1, QC3
4	t_{10}	qc_handling	67.0	1	AGV 1, QC3
5	t_9	qc_handling	64.0	1	AGV 1, QC3
6	t_8	qc_handling	72.0	1	AGV 1, QC3
7	t_4	empty_leg	27.8	1	AGV 1, QC2
8	t_4	loaded_leg	41.7	1	AGV 1, QC2
9	t_4	qc_handling	30.0	1	AGV 1, QC2
travel / handling			201.39 / 233.00	$\rho = 0.464$	
sum of on-path durations			434.39	$= C_{\max}$	
<i>energy-optimal end: $C_{\max} = 728.25$ s, $E = 12,025$ kJ</i>					
0	t_0	empty_leg	0.0	1	AGV 0, QC1
1	t_0	loaded_leg	62.5	1	AGV 0, QC1
2	t_3	empty_leg	31.2	1	AGV 0, QC1
3	t_3	loaded_leg	62.5	1	AGV 0, QC1
4	t_4	empty_leg	41.7	1	AGV 0, QC2
5	t_4	loaded_leg	62.5	1	AGV 0, QC2
6	t_{10}	empty_leg	31.2	1	AGV 0, QC3
7	t_{10}	loaded_leg	83.3	1	AGV 0, QC3
8	t_{11}	empty_leg	31.2	1	AGV 0, QC3
9	t_{11}	loaded_leg	62.5	1	AGV 0, QC3
10	t_9	empty_leg	31.2	1	AGV 0, QC3
11	t_9	loaded_leg	62.5	1	AGV 0, QC3
12	t_8	empty_leg	31.2	1	AGV 0, QC3
13	t_8	loaded_leg	62.5	1	AGV 0, QC3
14	t_8	qc_handling	72.0	1	AGV 0, QC3
travel / handling			656.25 / 72.00	$\rho = 0.901$	
sum of on-path durations			728.25	$= C_{\max}$	

4.3 The degenerate regime

Table 4.5 applies the method to both front extremes of all ten large dual-cycling instances, at up to 160 tasks. The decomposition closes on all 50 replicates.

Migration is near-absent throughout: mean absolute migration is 0.031, and the transport share lies between 0.87 and 0.97 at both ends of every front. Considered in isolation this constitutes a negative result for the hypothesis that the bottleneck migrates.

The explanation lies in the third column. Every one of the ten published large instances has $A/Q = 0.50$ exactly, twice as many cranes as vehicles. With vehicles thus scarce, the schedule is transport-bound wherever it lies on the front: slowing it to conserve energy slows the vehicles, which already constituted the binding resource, so no transfer occurs. There is no bottleneck to migrate because only one candidate was ever available. This motivates the sweep instead of

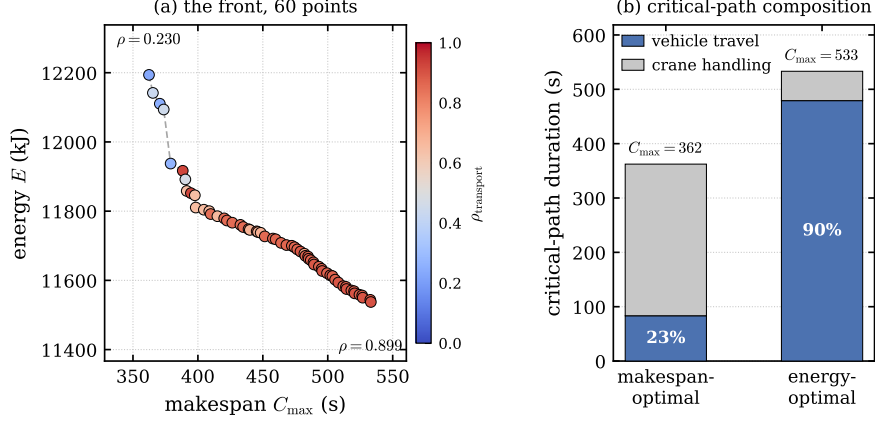


Figure 4.1: One explained front, for L21 at $A/Q = 1.67$ and seed 0, whose ten-seed cell mean appears in Table 4.6. (a) All sixty non-dominated points on the objective plane, each shaded by its duration-weighted transport share, so that the figure reports not only the trade-off but what binds at each compromise. The makespan-optimal end is crane-bound ($\rho_{\text{transport}} = 0.230$) and the energy-optimal end vehicle-bound ($\rho_{\text{transport}} = 0.899$); the transition between them occurs across the front rather than at its ends. (b) The critical path of each extreme resolved into its travel and handling parts, which sum to that end’s makespan by Eq. (2.6). The share, not the objective value, is what the explanation reports.

Table 4.5: Exact explanation of both front extremes on the ten large dual-cycling instances, averaged over five seeds each. Every instance has $A/Q = 0.50$, and the transport share is high at both ends, so migration is negligible throughout. Migration is the makespan-optimal share minus the energy-optimal share, as in Table 4.6. This is the degenerate regime that motivates the sweep in Table 4.6. The final two columns are the learned surrogate’s own migration on the same fronts and the Jaccard agreement of its recovered critical leg set with the exact one, so the model is measured on the largest instances in the study rather than only on the small ones. The decomposition closes on all 50 replicates, so none is excluded.

Instance	N	A/Q	n	exact		exact share		surrogate	
				$C_{\max}^{\text{mk}}$	$C_{\max}^{\text{en}}$	ρ_{mk}	ρ_{en}	migration	leg J
DL01	40	0.50	5	1799.8	3633.8	0.963	0.874	+0.089	0.960
DL03	50	0.50	5	2311.3	3754.3	0.971	0.890	+0.100	0.909
DL05	60	0.50	5	3183.3	5280.0	0.967	0.948	+0.019	0.908
DL07	70	0.50	5	3765.7	6121.1	0.959	0.909	+0.046	0.917
DL02	80	0.50	5	4178.1	6516.3	0.941	0.905	+0.036	0.966
DL09	80	0.50	5	3916.9	5569.6	0.959	0.954	+0.023	0.868
DL04	100	0.50	5	5364.7	7897.7	0.951	0.929	+0.025	0.920
DL06	120	0.50	5	6977.5	10763.8	0.955	0.955	−0.000	0.927
DL08	140	0.50	5	8377.2	12595.0	0.956	0.951	−0.000	0.944
DL10	160	0.50	5	8511.5	13456.2	0.972	0.968	+0.005	0.924
Mean migration +0.031, mean migration 0.031									

excusing the result, and it yields a testable prediction: migration should appear once vehicles are sufficiently numerous for cranes to become competitive.

Figure 4.2 presents both components of this account on the published set. Panel (a) reports the two ends of every DL front. The transport share exceeds 0.87 at the energy-optimal end and 0.94 at the makespan-optimal end of all ten instances, so the vehicles bind at either extreme and the separation between the two markers, which is the migration, is correspondingly small. Panel (b) places the set beside the small instances. At the common ratio the two sets are statistically indistinguishable, at +0.031 and −0.002 respectively, against −0.404 for the small instances at higher ratios. The published set is therefore not a family in which migration fails to occur, but a family observed at a single fleet ratio that lies below the threshold.

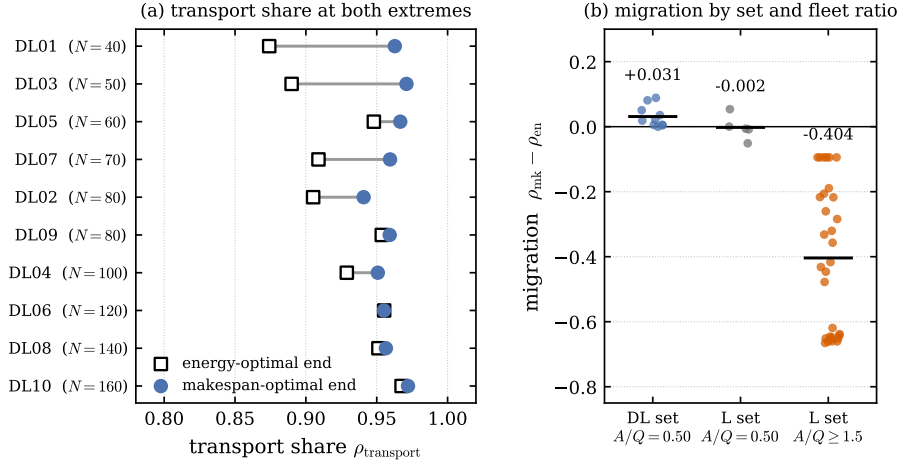


Figure 4.2: The published large dual-cycling set, over five seeds per instance. (a) Transport share at the two ends of every DL front, ordered by instance size. Both ends lie above 0.87, and the separation between them is the migration. (b) Migration for the DL set at its native $A/Q = 0.50$, for the small instances at the same ratio, and for the small instances at $A/Q \geq 1.5$, with group means marked. The first two groups are comparable in magnitude and the third is not. The decomposition closes on all 50 DL replicates.

Table 4.6: Bottleneck migration across the fleet size, for the six swept instances. Migration is the signed shift of the duration-weighted transport share, $\rho_{mk} - \rho_{en}$, on the exact critical path, each cell the mean over ten seeds. Beneath each cell the model’s own migration is given, and beneath that the ratio A/Q followed by the mean absolute difference $|\Delta\rho|$ between the two. The ratio differs between instances because the crane count does. Parenthesised cells are those not significant at $\alpha = 0.05$ under a one-sample Wilcoxon signed-rank test against zero, Holm-corrected within each instance’s nine fleet sizes. Migration is negligible left of $A/Q \approx 1$ and saturates to its right.

Instance	fleet size A , with A/Q beneath								
	1	2	3	4	5	6	7	8	9
L01	−0.051	+0.108	−0.432	−0.094	−0.094	−0.094	−0.094	−0.094	−0.094
model	−0.051	+0.100	−0.432	−0.094	−0.095	−0.095	−0.095	−0.095	−0.096
	0.50, 0.001	1.00, 0.012	1.50, 0.001	2.00, 0.001	2.50, 0.001	3.00, 0.001	3.50, 0.001	4.00, 0.001	4.50, 0.001
L07	(+0.000)	−0.088	−0.660	−0.648	−0.654	−0.651	−0.638	−0.643	−0.619
model	−0.000	−0.104	−0.606	−0.648	−0.611	−0.652	−0.574	−0.561	−0.615
	0.50, 0.001	1.00, 0.017	1.50, 0.055	2.00, 0.001	2.50, 0.044	3.00, 0.002	3.50, 0.065	4.00, 0.083	4.50, 0.006
L15	−0.008	(+0.056)	−0.321	−0.357	−0.260	−0.217	−0.217	−0.189	−0.206
model	−0.008	+0.044	−0.254	−0.353	−0.239	−0.213	−0.155	−0.186	−0.177
	0.50, 0.000	1.00, 0.013	1.50, 0.156	2.00, 0.004	2.50, 0.022	3.00, 0.004	3.50, 0.062	4.00, 0.003	4.50, 0.061
L21	−0.235	−0.247	−0.658	−0.664	−0.662	−0.661	−0.651	−0.665	−0.646
model	−0.203	−0.239	−0.470	−0.660	−0.655	−0.656	−0.647	−0.662	−0.646
	0.33, 0.032	0.67, 0.028	1.00, 0.187	1.33, 0.004	1.67, 0.007	2.00, 0.005	2.33, 0.005	2.67, 0.003	3.00, 0.007
L28	−0.002	(+0.054)	−0.063	(−0.042)	−0.160	−0.332	−0.417	−0.478	−0.446
model	−0.002	+0.056	−0.050	−0.042	−0.140	−0.327	−0.413	−0.448	−0.444
	0.25, 0.001	0.50, 0.003	0.75, 0.036	1.00, 0.001	1.25, 0.093	1.50, 0.042	1.75, 0.009	2.00, 0.032	2.25, 0.015
L35	−0.012	(+0.007)	(−0.006)	(+0.027)	(−0.021)	−0.076	(−0.109)	−0.257	−0.284
model	−0.012	+0.000	−0.006	+0.024	−0.031	−0.088	−0.078	−0.073	−0.261
	0.17, 0.000	0.33, 0.009	0.50, 0.001	0.67, 0.007	0.83, 0.019	1.00, 0.016	1.17, 0.031	1.33, 0.184	1.50, 0.025

4.4 Bottleneck migration across the fleet-to-crane ratio

Table 4.6 reports the sweep and Figure 4.3 plots it, one panel per instance. Migration differs significantly from zero in 45 of the 54 instance–fleet cells after Holm correction, and the pattern is consistent across instances: negligible at low ratios, rising sharply near unity, and saturating thereafter.

Forty-one of the 45 significant cells carry a Holm-corrected p of 0.0176, the smallest value a ten-sample Wilcoxon test can attain, with a rank-biserial effect size of -1.00 in all but seven

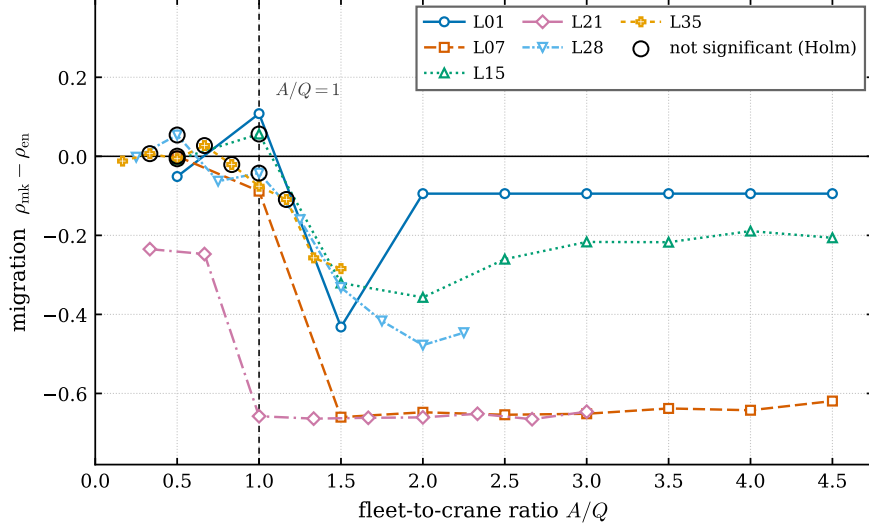


Figure 4.3: Migration against the fleet-to-crane ratio for the six swept instances, each cell the mean over ten seeds. Series are distinguished by marker and dash pattern so that they remain separable in greyscale. Open circles mark cells that fail significance after Holm correction; they are exactly the near-zero cells at low A/Q . The dashed vertical is $A/Q = 1$. The threshold structure is the result: migration is absent below $A/Q \approx 1$, grows sharply thereafter, and saturates. The crossing point is instance-dependent. The ratio ranges differ by instance because nine fleet sizes are swept against a fixed crane count that differs between them. Dispersion is omitted here and reported per cell in Table 4.6.

cells; the test therefore separates the two regimes without discriminating within either. At the lowest ratios migration is negligible. L07 at $A/Q = 0.50$ yields $+0.000$ and is one of the nine cells failing significance, every one of which lies at $A/Q \leq 1.17$, and L15, L28 and L35 all yield values within 0.06 of zero below $A/Q \approx 1$. This reproduces, from independent runs, the regime of Section 4.3 at the same ratio, which is the confirmation that section predicted.

Migration then increases sharply and saturates. L07 moves from -0.088 at $A/Q = 1.00$ to -0.660 at 1.50, thereafter remaining near -0.64 for every larger fleet. L21 attains -0.658 by $A/Q = 1.00$ and holds within 0.02 of -0.66 across the remaining six ratios, peaking at -0.665 . Taking the crossing as the first ratio at which an instance reaches half its own saturation, L21 crosses earliest at $A/Q = 1.00$, L35 at 1.33, and L01, L07, L15 and L28 all at 1.50. The threshold is therefore instance-dependent, not universal, which is to be expected: whether cranes can become binding depends on handling times and layout geometry, not on the ratio alone.

The sign identifies the direction of the transfer. Where migration is substantial it is negative, so by Eq. (3.3) the energy-optimal end is the travel-heavy one and the bottleneck moves towards vehicle travel as energy is prioritised. The worked case of Figure 4.1 makes this concrete on a single replicate of that cell: at the makespan-optimal end of L21 at $A/Q = 1.67$ the critical path is only 23.0% travel and therefore predominantly crane handling, whereas at the energy-optimal end it is 89.9% travel, a migration of -0.669 against the cell mean of -0.662 in Table 4.6.

The mechanism follows from the speed selection. Minimising the makespan drives every leg to the fastest available speed, which compresses travel until the crane chains, whose handling times are fixed and independent of vehicle speed, become the binding resource. Minimising energy does the reverse, selecting slow speeds that lengthen every leg until travel dominates the path again. The cranes therefore bind at the fast end and the vehicles at the slow end, which is why the transfer requires enough vehicles for the cranes to be competitive in the first place, and why it vanishes below $A/Q \approx 1$.

Table 4.7: Migration on all 35 published small instances at their native fleet ratios, ten seeds each, grouped by ratio. This campaign is independent of the sweep of Table 4.6, which varies the ratio on six instances; here the ratio is whatever the published configuration specifies. The proportion of replicates whose migration exceeds 0.1 in magnitude rises monotonically with the ratio, which corroborates the dependence on a larger set of instances. The ratios reach only 1.00, so the campaign probes the sub-threshold side alone. The decomposition closes on all 350 replicates, so none is excluded.

A/Q	n	mean migration	mean $ \text{migration} $	$ \text{migration} > 0.1$
0.33	10	+0.007	0.036	0%
0.40	40	+0.025	0.049	12%
0.50	60	−0.010	0.062	13%
0.67	80	−0.071	0.126	41%
1.00	160	−0.110	0.172	56%
Overall: $n = 350$, mean -0.065 , mean $ \text{migration} $ 0.124				

The single instance exhibiting the opposite sign, L01 at $A/Q = 1.00$ with +0.108 inverted relative to its neighbours, is the four-task instance, whose 35-point front offers too few structurally distinct schedules for the argument to apply.

A second campaign, independent of the sweep, holds each of the 35 published small instances at the fleet ratio its own configuration specifies and runs ten seeds of each. Table 4.7 groups its 350 replicates by ratio. The proportion whose migration exceeds 0.1 in magnitude rises monotonically from none at $A/Q = 0.33$ to 56% at $A/Q = 1.00$. Because the published configurations reach only $A/Q = 1.00$, this campaign observes the sub-threshold side alone, and it corroborates the ratio dependence on the whole set and no longer on the six instances of the sweep alone. It also shows that the onset is gradual from about $A/Q = 0.4$ rather than a step at unity.

Several cells report a standard deviation of exactly zero across ten seeds, notably L01 throughout and L21 at the lowest ratio. In those cells every seed recovers the same two extreme schedules, which is unsurprising for a four-task instance and indicates that the search has saturated.

4.5 The three instance families

Every result to this point rests on the loading family: in the 35 published small instances each of the 368 container moves is a loading task, so the unloading recurrence (2.4) is never exercised, and the dual-cycling instances of Section 4.3 are observed at a single fleet ratio. The source supplies the remedy without further data. Loading instances are Table 5 as printed; unloading instances are obtained from the same table by reversing the origins and destinations, so that a container originates at the crane and is delivered to the yard station; and a mixed instance is formed by combining one loading instance with one unloading instance. Applying these two constructions to Table 5 yields 35 unloading and 26 mixed instances alongside the 35 loading instances already used, on the same layout and with no value retyped.

The pairing that forms the mixed set is stated in the source: the “comb. of” column of Table 3 names, for each of the 26 instances, the unloading and loading instance it combines. That column is used as printed, so the family reproduces the published set instead of approximating it. It pairs by neither adjacency nor position, D04 combining U01 with L02 and D06 combining U02 with L01, and the reconstruction is confirmed against an independent published quantity: the task count of all 26 agrees with the Q–T column of the same table.

Each of the 96 instances was run at nine fleet sizes with ten seeds, and the transport share was

Table 4.8: Migration by fleet-to-crane ratio on the three instance families. Loading is Table 5 as printed; unloading is the same data with origins and destinations reversed; mixed combines one loading with one unloading instance, following the construction stated in the source. Each cell is the mean over all instances of that family and ten seeds, restricted to the ratios every family attains. The threshold is reproduced independently on all three: migration is negligible below $A/Q \approx 1$ and saturates near -0.5 above it. Family sizes: loading 3,150 replicates and 232,835 front points; unloading 3,150 replicates and 230,211 front points; mixed 2,340 replicates and 194,853 front points.

A/Q	loading	unloading	mixed
0.25	-0.021	-0.014	-0.019
0.33	-0.043	-0.042	+0.205
0.50	-0.049	-0.022	+0.095
0.67	-0.060	-0.023	+0.075
0.75	-0.083	-0.017	-0.115
1.00	-0.106	-0.017	-0.153
1.25	-0.279	-0.166	-0.545
1.33	-0.359	-0.207	-0.197
1.50	-0.453	-0.291	-0.386
1.67	-0.527	-0.372	+0.005
1.75	-0.490	-0.400	-0.562
2.00	-0.497	-0.469	-0.410
2.25	-0.514	-0.448	-0.555
2.33	-0.538	-0.538	-0.179
2.50	-0.488	-0.483	-0.342
2.67	-0.550	-0.530	-0.179
3.00	-0.503	-0.480	-0.409
3.50	-0.500	-0.447	-0.407
4.00	-0.495	-0.445	-0.412
4.50	-0.473	-0.445	-0.409

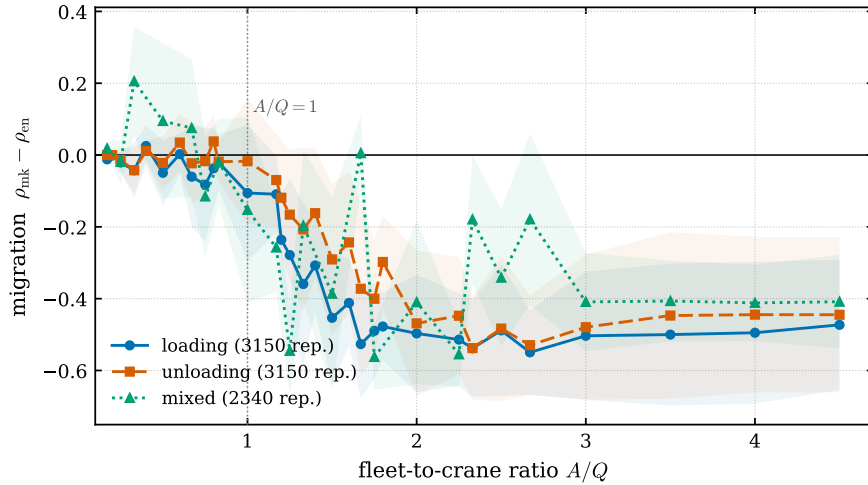


Figure 4.4: Migration against the fleet-to-crane ratio on the three instance families, with the shaded band one standard deviation across instances and seeds. The threshold is reproduced independently on each: migration is negligible below $A/Q \approx 1$, turns sharply negative through 1.2 to 1.5, and saturates near -0.5 . The mixed family departs from the other two below the threshold, where its migration is consistently positive rather than near zero.

computed at every point of every resulting front. The campaign comprises 8,640 replicates and 657,899 front points; no replicate failed to complete and the decomposition closes at every point. Table 4.8 reports migration by ratio for each family and Figure 4.4 plots it.

The threshold established on loading instances therefore holds on all three families. Two differences are visible and neither follows from the mechanism of Section 4.4. Above the threshold the unloading family migrates no further than the loading family at any ratio, the gap running from 0.09 to 0.16 between $A/Q = 1.25$ and 1.75 and closing to zero at 2.33; the recurrences differ in where the handling term enters the maximum, which is a candidate explanation that

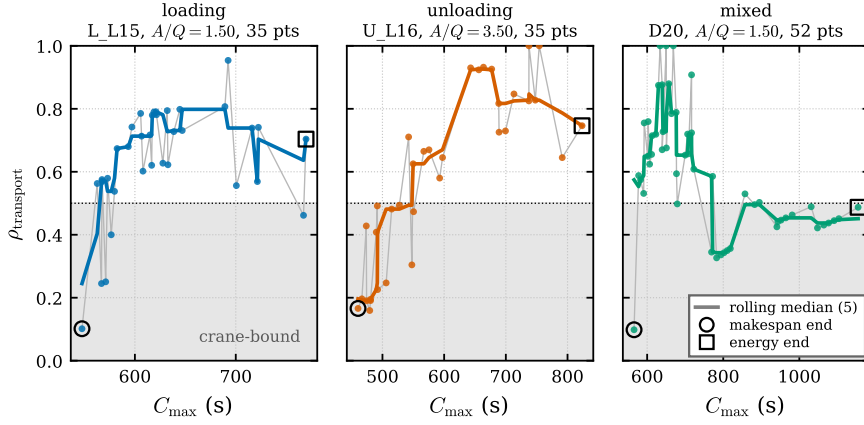


Figure 4.5: The transport share at every point of one front from each family, plotted against the makespan of that point. The dotted rule at $\rho_{\text{transport}} = 0.5$ separates the crane-bound from the vehicle-bound regime. The share rises from the makespan-optimal end to the energy-optimal end in all three, but not monotonically.

was not tested. Below the threshold the mixed family carries a positive migration of up to +0.205 where the single-cycling families sit near zero, so on a mixed instance with few vehicles the makespan-optimal end is the more travel-heavy of the two. Both are reported as measured regularities and neither is claimed as understood.

Because the share is computed at every front point, the explanation can also be read as a profile along the front. Figure 4.5 shows one front from each family. The transition from a crane-bound to a vehicle-bound critical path occurs across the front rather than at its ends, and it is not monotone: neighbouring points differ in which resource binds, with the share moving by more than 0.3 between adjacent compromises. A two-point measurement reports the endpoints of this profile and is silent about its shape. Whether the non-monotonicity reflects genuine structure or the search returning structurally distinct schedules at neighbouring objective values is not resolved here.

4.6 How the explanation changes during the search

Every result to this point explains finalised solutions. Two questions remain open, and they are the two roles the surrogate can take beyond post-hoc explanation. Whether the explanation of a front is settled once the front is, or still moving while the search converges; and whether the learned model tracks that movement, on which any later use of it inside the search would depend.

Both are answered by snapshotting the solver’s archive every ten generations. Six instances from each of the three families are run at three fleet sizes with five seeds, giving 54 cells, 270 replicates and 2,970 snapshots. At every snapshot the transport share is computed twice: once from the exact recurrence, and once from the surrogate, whose gradients are weighted by its own predicted durations so that the two explanations are never mixed. One model is fitted per instance and fleet size and reused across seeds, since it depends on the instance alone. No replicate failed.

Figure 4.6 reports the first. The explanation is not settled early: above the threshold the mean migration moves from -0.101 at generation zero to -0.274 at generation one hundred, reaching its final value to within 0.02 only by generation forty; below the threshold it moves from

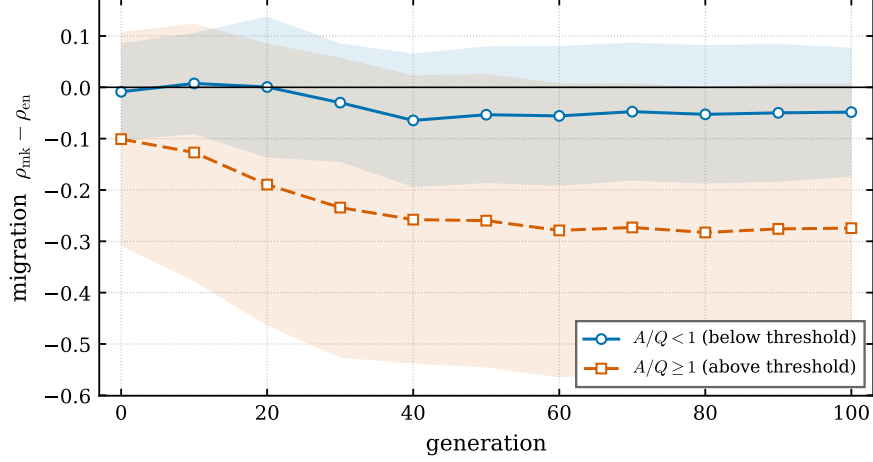


Figure 4.6: Migration against generation, over 270 replicates with the solver’s archive snapshotted every ten generations, grouped by side of the fleet-to-crane threshold with the band one standard deviation over replicates. Above the threshold the explanation moves from -0.101 to -0.274 and reaches its final value to within 0.02 only by generation forty; below it there is little to settle. An explanation read from an unconverged front is therefore biased towards zero.

-0.009 to -0.048 and settles by generation thirty. An explanation read from an unconverged front therefore understates the effect by more than half, because the early archive holds few structurally distinct schedules and its two extremes lie close together. The observation is a caution on procedure: it establishes that explaining finalised solutions, as Section 4.4 does, is the defensible use, and it gives the feedback loop of Chapter 5 a precondition, since guidance drawn from an early generation would act on an explanation that has not converged.

The surrogate tracks that movement closely. Over all 2,970 snapshots the median absolute difference between the model’s migration and the exact value is 0.0026, and the model is therefore reproducing the explanation while the search is still running. The agreement is family-dependent and its error distribution is skewed: the median is 0.0014 on loading, 0.0052 on unloading and 0.0062 on mixed instances, but the corresponding means are 0.022, 0.052 and 0.076, and the ninetieth percentile reaches 0.347 on the mixed family. The model is thus accurate in the typical case and occasionally far from it, which is the same pattern Section 4.7 reports for finalised solutions and has the same cause: where several resources are nearly balanced, a small error in a predicted duration changes which activity binds.

A second consequence concerns the generation budget. Chapter 5 notes that all runs use 100 generations against the published 300; the measurements here show migration settled well before the hundredth generation on both sides of the threshold, so the reduced budget is sufficient for this particular measurement, whatever it costs in absolute front quality.

4.7 Reproduction by the surrogate

Table 4.9 reports the secondary tier: whether the learned model reproduces the explanation that the max-plus layer supplies exactly.

Calibration is high, with R^2 on the makespan at 0.999 or above. The energy figure should not be read as a success, for the architectural reason given in Section 3.4.

The meaningful quantity is the reproduction of ρ . The mean absolute difference between the surrogate’s migration estimate and the exact value is 0.026 across the sweep, against migration

Table 4.9: Surrogate calibration and post-hoc faithfulness over the sweep (mean over all fleet sizes and seeds per instance). These quantities belong to the secondary, over-requirements tier: they measure how well the model reproduces an explanation that the exact oracle already provides without it.

Instance	n	$R^2(C_{\max})$	$R^2(E)$	Leg Jaccard	$ \Delta\rho $
L01	90	0.9991	1.0000	0.990	0.0022
L07	90	0.9993	1.0000	0.882	0.0304
L15	90	0.9995	1.0000	0.917	0.0362
L21	90	0.9994	1.0000	0.885	0.0309
L28	90	0.9994	1.0000	0.888	0.0258
L35	90	0.9993	1.0000	0.825	0.0326

magnitudes reaching 0.66, a relative error of approximately five per cent on the effect being measured. On the cells of largest migration the discrepancy is also largest, reaching 0.187 on L21 at $A/Q = 1.00$, so the reproduction is weakest precisely where the phenomenon is strongest. That anti-correlation qualifies the secondary result, and is the reason the two tiers are reported separately throughout. The exact explanation of Tables 4.3–4.6 is unaffected by the surrogate’s error.

4.8 Discussion

The results support a specific and bounded claim. For this problem the reason a solution is Pareto-optimal can be stated exactly, in seconds attributable to named physical activities, with the attribution summing to the objective by construction. This is stronger than a feature attribution over decision variables, and it is available because the objective possesses (max,+) structure that the max-plus layer preserves; no technique imported from machine learning supplies it. The architectural lesson is narrow but transferable: the encoder was not the obstacle to faithful attribution, the pooled regression head was, and replacing that head with the problem’s own recurrence removes the obstacle without sacrificing the learned representation.

The explanation is not static along the front. Where vehicles are sufficiently plentiful, trading time for energy transfers the binding constraint from the cranes to the vehicles by an amount reaching two-thirds of the critical path; where they are scarce, as in every published large instance, it does not, there being no second candidate for the bottleneck. An operator reading a front therefore cannot assume its members are limited by the same resource, and A/Q indicates which regime obtains. The surrogate reproduces that explanation to a mean absolute ρ error of 0.026, with critical-leg agreement of 0.898 on the small instances and 0.924 on the published large ones, and it does so least accurately where the effect is largest, since Pareto-optimal schedules are those in which resources are nearly balanced. The exact recurrence remains the yardstick, but it can only be applied to a schedule that has already been evaluated; the model predicts the durations and can therefore explain a candidate before that evaluation exists, which is the setting the feedback loop of Chapter 5 would require.

Limitations and Future Work

Non-uniqueness of the critical path. The makespan is a maximum over several terminal nodes, and on a schedule where two or more attain it there is more than one critical path. Any single maximiser of a maximum is a valid subgradient, whereas an average of several is not a valid selection for an additive decomposition: dividing the gradient between tied terminals gives every activity on both paths a fractional share, and the durations then no longer sum to the makespan. The implementation therefore selects the maximal terminal by index at every reduction, including the batched form, which is the convention the recurrence of Section 3.4 already follows. The attribution reported for such a schedule is consequently one critical path among several rather than the set of them, and ρ is the share of the path selected. Under that convention the decomposition closes on all 940 replicates and at all 657,899 front points.

Scope of the claim. All runs use 100 generations against the published 300, which suffices for a measurement depending on the two front extremes but places absolute front quality below what the method attains, so no claim about front quality should be drawn here. The sweep establishes that migration depends on A/Q and locates the transition between 1.0 and 1.5 across the six swept instances, but does not predict it from instance parameters. It is also established on the small-instance layout alone: the large dual-cycling instances describe a twenty-crane terminal whose distance matrix is not interchangeable, and are observed only at $A/Q = 0.50$, so extending the threshold to them is an extrapolation across layouts. Sweeping the fleet on three large dual-cycling instances would settle it, at roughly forty core-hours. Finally, the surrogate’s agreement with the exact critical set is lowest where migration is largest, since Pareto-optimal schedules are those in which resources are nearly balanced and small prediction errors alter which activity binds; this bounds the secondary tier only.

Further work. Two implemented components are excluded. A power-coupled regime, in which a terminal-wide budget couples the vehicles, is the one setting in which the surrogate would predict a quantity with no closed form, the waiting time induced by power contention, and therefore the one setting that requires a learned component; it is the strongest candidate for future work but needs a separate problem statement, originating as it does in a different literature. Its three-velocity configuration and power coefficients are retained here, belonging to the original problem. The natural continuation is to feed the explanation back into the search, which divides into two steps of unequal size. The smaller, comparing how the explanation changes across generations, is carried out in Section 4.6 and shows that an explanation read from an unconverged front is biased towards zero. The larger is attribution-guided search proper, biasing mutation towards the activities currently on the critical path, in the spirit of decision-focused learning [9].

Chapter 6

Conclusion

This thesis set out to explain why particular solutions to a bi-objective container-terminal scheduling problem are Pareto-optimal. Its one core idea is that such a schedule can be explained exactly, in seconds attributable to named physical activities, and that a learned model can produce the same explanation. The instances are the published ones throughout: the loading set as printed, the unloading set reversed from it, and the mixed set formed by the pairing the source states, each reconstruction checked against the optima published for it before any result was drawn from it.

Because the makespan is a maximum of sums of activity durations, it is a longest path over the $(\max, +)$ semiring, and the sensitivity of the makespan to each activity is exactly zero or one. A conventional surrogate cannot express this: a pooled regression head produces a dense Jacobian and therefore assigns credit to provably irrelevant activities. Replacing that head with a differentiable max-plus dynamic programme, whose backward pass routes the subgradient through the recorded argmax edge, restores the property, and differentiating the composition returns the critical path itself. From that decomposition the duration-weighted critical share ρ summarises an explanation as one number, the fraction of the makespan spent moving on the vehicles.

The empirical result is that the bottleneck migrates, and that whether it does is governed by the fleet-to-crane ratio. Across 540 statistically tested replicates migration is significantly non-zero in 45 of 54 instance–fleet cells and attains a magnitude of 0.665; below $A/Q \approx 1$ it vanishes, both front extremes being transport-bound. A further 8,640 replicates reproduce the threshold on three instance families, so the result does not depend on every container being a loading move. Where it occurs the cranes bind at the makespan-optimal end and the vehicles at the energy-optimal end, which accounts for its absence on the ten published large instances, each having $A/Q = 0.50$. The operational reading is that two schedules on one front need not be limited by the same resource, and that the fleet ratio indicates whether an operator is in the regime where this distinction matters.

The learned surrogate reproduces the same explanation to a mean absolute ρ error of 0.026, with critical-leg agreement of 0.898 on the small instances and 0.924 on the largest published ones at up to 160 tasks. That is what licenses the method beyond the cases examined here: the exact recurrence is available for a resolved schedule, but it requires one, whereas the model predicts the durations and can therefore explain a candidate that has not been evaluated. Every setting in which the explanation is wanted before evaluation, or in which no closed form exists, rests on the learned component.

Bibliography

- [1] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [2] François Baccelli, Guy Cohen, Geert Jan Olsder, and Jean-Pierre Quadrat. *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Wiley, 1992.
- [3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.
- [4] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [5] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.
- [6] Jacques Carlier and Éric Pinson. An algorithm for solving the job-shop problem. *Management Science*, 35(2):164–176, 1989.
- [7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [8] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [9] Adam N. Elmachtoub and Paul Grigas. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022.
- [10] Dalila B. M. M. Fontes and S. Mahdi Homayouni. Energy-efficient scheduling of intraterminal container transport. In *Advances in Intelligent Transportation Systems*, pages 155–182, 2022.
- [11] Dalila B. M. M. Fontes and S. Mahdi Homayouni. A bi-objective multi-population biased random key genetic algorithm for joint scheduling of quay cranes and speed adjustable vehicles in container terminals. *Flexible Services and Manufacturing Journal*, 35:241–268, 2023.
- [12] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.
- [13] José Fernando Gonçalves and Mauricio G. C. Resende. Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, 17:487–525, 2011.
- [14] Bernd Heidergott, Geert Jan Olsder, and Jacob van der Woude. *Max Plus at Work: Modeling and Analysis of Synchronized Systems*. Princeton University Press, 2006.
- [15] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [16] S. Mahdi Homayouni and Dalila B. M. M. Fontes. Joint scheduling of quay cranes and speed adjustable vehicles in container terminals. *International Journal of Production Research*, 2021.
- [17] Hisao Ishibuchi, Hiroyuki Masuda, Yuki Tanigaki, and Yusuke Nojima. Modified distance calculation in generational distance and inverted generational distance. In *Evolutionary Multi-Criterion Optimization (EMO)*, pages 110–125, 2015.
- [18] Dave S. Kerby. The simple difference formula: An approach to teaching nonparametric correlation. *Comprehensive Psychology*, 3, 2014.
- [19] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.
- [20] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [21] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2:56–67, 2020.
- [22] Marin Vlastelica Pogačić, Anselm Paulus, Vit Musil, Georg Martius, and Michal Rolínek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations (ICLR)*, 2020.
- [23] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1:206–215, 2019.
- [24] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [25] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference (WWW)*, pages 2022–2032, 2019.
- [26] Cong Zhang, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, and Chi Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [27] Eckart Zitzler and Lothar Thiele. Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999.